

Test Report

ECE 422C - Project 2, Phase II | EID: au4355

Reproduce the results

From project2_au4355, run `sh run-tests.sh`. The script compiles every Java source into build with UTF-8 and `-Xlint:all`, then runs all three suites. It stops on compilation errors or any failed check. Tests throw `AssertionError` directly; JVM assertions do not need enabling.

Observed result: 561 checks passed with no compiler warnings. `MasterMindRulesTest`: 11; `TestHarness`: 245; `WordleRulesTest`: 305. The 400 randomized cases use fixed seed 422 and independent scoring oracles. These counts describe local automated checks, not hidden-grader results or a claim of exhaustive correctness.

1. Required Wordle example and excess letters

Secret APPLE, guess ALLEY: C P A P A. The first L receives PRESENT; the second receives ABSENT because APPLE has only one L. The A is CORRECT, E is PRESENT, and Y is ABSENT. This is tested against the user-provided dictionary.

2. Exact match must consume its occurrence first

Secret BREAD, guess BLEED: C A C A C. The first guessed E is CORRECT. The second E is ABSENT because BREAD has only one E; it must not receive PRESENT. Secret APPLE, guess PAPAL gives P P C A P in a test-only fixture, confirming that a later exact P is reserved before an earlier misplaced P is considered.

3. Other repeated-letter cases

Secret SHORE, guess CHESS: A C P P A; only one of two guessed S letters receives PRESENT. Secret BLOOM, guess BROOM: C A C C C; both O occurrences are correctly matched. Secret BREED, guess BLEED: C A C C C; both E occurrences are exact. Additional fixtures check LLAMA against APPLE and LEVEL against ELDER.

4. Ordered feedback, wins, and absence

APPLE against APPLE yields C C C C C and `isWinningGuess` is true. BRICK against HOUSE yields A A A A A and does not win. Each targeted scoring check verifies the original guess, exact feedback text, and winning status.

5. Independent randomized Wordle scoring

Two hundred generated guess/secret pairs use lengths 1-9 and a three-letter alphabet to create many duplicates. Each pair is placed in a test-only allowed set. The production scorer counts unmatched secret letters; the oracle instead tracks and consumes individual secret positions after reserving exact matches. Every position and the winning status must agree. Synthetic strings in test fixtures are not part of the production dictionary.

6. Word validation and configuration

Reject null, empty, short, long, nonalphabetic, embedded-space, lowercase rules-API, accented, and absent-from-dictionary guesses. Invalid secrets are rejected too. Console input is separately checked for trimming and uppercase normalization. Nonpositive word length/attempt limits and empty or malformed in-memory dictionaries are rejected. A three-letter, two-attempt CAT/ACT game verifies changed configuration: ACT gives P P C, then CAT wins.

7. Dictionary loading and isolation

The production dictionary loads exactly the 50 supplied words, with APPLE first. Temporary files test UTF-8 BOM handling, lowercase conversion, duplicate removal, comments, blank lines, mixed word lengths, malformed entries, empty files, no matching-length entries, and missing files. Temporary fixtures are deleted after use. Modifying the original collection cannot change the rules' dictionary; its exposed set is unmodifiable. Feedback arrays are copied on construction and access.

8. Invalid input, attempts, and history

Script HISTORY, ZZZZZ, ALLEY, GRAPE, HISTORY, APPLE against test secret APPLE. The first three prompts all show six attempts remaining. Only valid words consume attempts. History is exactly ordered: 1. ALLEY -> C P A P A; 2. GRAPE -> A A P P C. Empty history has a clear message. The final lowercase, padded apple input is normalized and wins.

9. Win, loss, and replay

Six ALLEY guesses lose and reveal APPLE. Five ALLEY guesses followed by APPLE win on the last valid attempt without printing a loss. Winning, entering an invalid replay answer, then YES starts a fresh round with empty history. NO exits. A one-attempt game verifies replay after losing. EOF during a game and at replay terminates cleanly. Normal mode with a one-word dictionary wins without a test-mode reveal.

10. Secret selection, locale, and launch behavior

Twenty seeded selections agree with Random.nextInt over the loaded dictionary. Test mode always uses the first eligible word. Turkish-locale lowercase console commands and guesses still work. Driver launches both games, rejects invalid arguments, accepts an external dictionary path override, and reports a missing dictionary. Mastermind still launches when the Wordle dictionary is missing. The original three-argument GuessRecord constructor remains usable.

11. Mastermind regression coverage

The unchanged original suite verifies exact, misplaced, absent, and duplicate peg feedback; invalid colors and length; and a five-peg configuration. Examples: RBRB against BRRR gives 1B_2W; BBRR against RRBB gives 0B_4W; RGBRG against RGBGR gives 3B_2W.

The unchanged harness adds 200 seeded scoring cases across 1-10 pegs and 1-10 colors. Its frequency-count oracle independently computes total color matches and subtracts exact matches. Forty-five additional checks cover configuration errors, invalid guesses/secrets, history, attempts, wins, losses, replay, EOF, changed configuration, test disclosure, and locale handling.

Packaging and portability

The final ZIP is extracted into a fresh temporary directory, compiled, and tested using the included external dictionary. It excludes stale compiled classes and the extensionless GuessingGame draft. Java sources also compile against the Java 8 API using `javac --release 8`. Reports are rendered to page images and visually checked before delivery.